

Java + Maven + Spring Boot — CI/CD Cheat Sheet

Quick reference for DevOps engineers working with Java/Maven-based projects

Core Config Files

pom.xml	Project-level build file (committed to Git): defines dependencies, plugins, build steps, and project metadata.
settings.xml	Machine/user-level config (~/.m2/settings.xml, NOT committed): repository URLs, mirrors, and credentials for private repos like Nexus/Artifactory.

Maven Build Lifecycle Commands (CI/CD)

Command	What it does	Use in CI/CD
<code>mvn clean</code>	Deletes target/ (old build output)	Start of every pipeline stage
<code>mvn compile</code>	Compiles src/main/java only	Rarely used alone
<code>mvn test</code>	Compiles + runs unit tests	Fast feedback / PR checks
<code>mvn clean verify</code>	Compile + test + integration checks; produces jar in target/	Standard CI build/test stage
<code>mvn clean package</code>	Same as verify, plus builds the .jar	When you just need the artifact
<code>mvn clean install</code>	package + copies jar to local ~/.m2 repo	Local dev / multi-module builds (rarely needed in CI)
<code>mvn deploy</code>	Publishes jar to remote repo (Nexus/Artifactory)	CD stage — the real, shared artifact

Run the Application & Tests

Run app (dev)	<code>mvn spring-boot:run</code>
Run app (built jar)	<code>java -jar target/<artifact>-<version>.jar</code>
Run all tests	<code>mvn test</code>
Run one test class	<code>mvn test -Dtest=ClassName</code>
Run one test method	<code>mvn test -Dtest=ClassName#methodName</code>
Skip tests (not recommended)	<code>mvn install -DskipTests</code>

Typical CI/CD Pipeline Flow

1. Checkout code → 2. Restore ~/.m2 cache (*speed only — not the deployable artifact*) → 3. **mvn clean verify** (compile + test, artifact lands in target/) → 4. **mvn deploy** to Nexus/Artifactory, or Docker build using target/*.jar → 5. Save ~/.m2 cache for next run

Key point: target/ = build output for one project (wiped by clean). ~/.m2/repository = local dependency cache + install target, ephemeral on most CI agents. Nexus/Artifactory (via mvn deploy) = the permanent, shared home for your artifact — that's what other environments/teams should pull from, not a CI agent's local .m2.